In [1]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [3]:
```python
df=pd.read_csv("C:/Users/userp/Downloads/diabetes.csv")
```

In [4]:
```python
df
```

Out[4]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPe
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
...	...	...	...	...	...	...	
763	10	101	76	48	180	32.9	
764	2	122	70	27	0	36.8	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

768 rows × 9 columns

In [16]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

In [17]:
```python
df.describe()
```

Out[17]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	B
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100

In [18]:
```
df.columns
```

Out[18]:
```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insu
lin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')
```

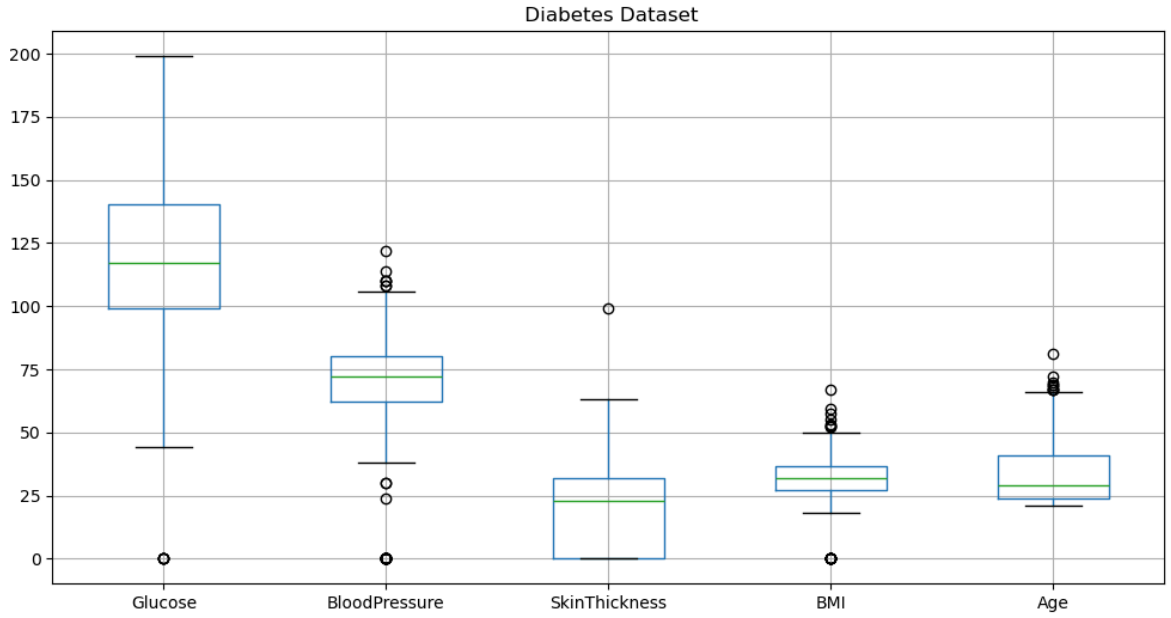
In [19]:
```
df.shape
```

Out[19]:
```
(768, 9)
```

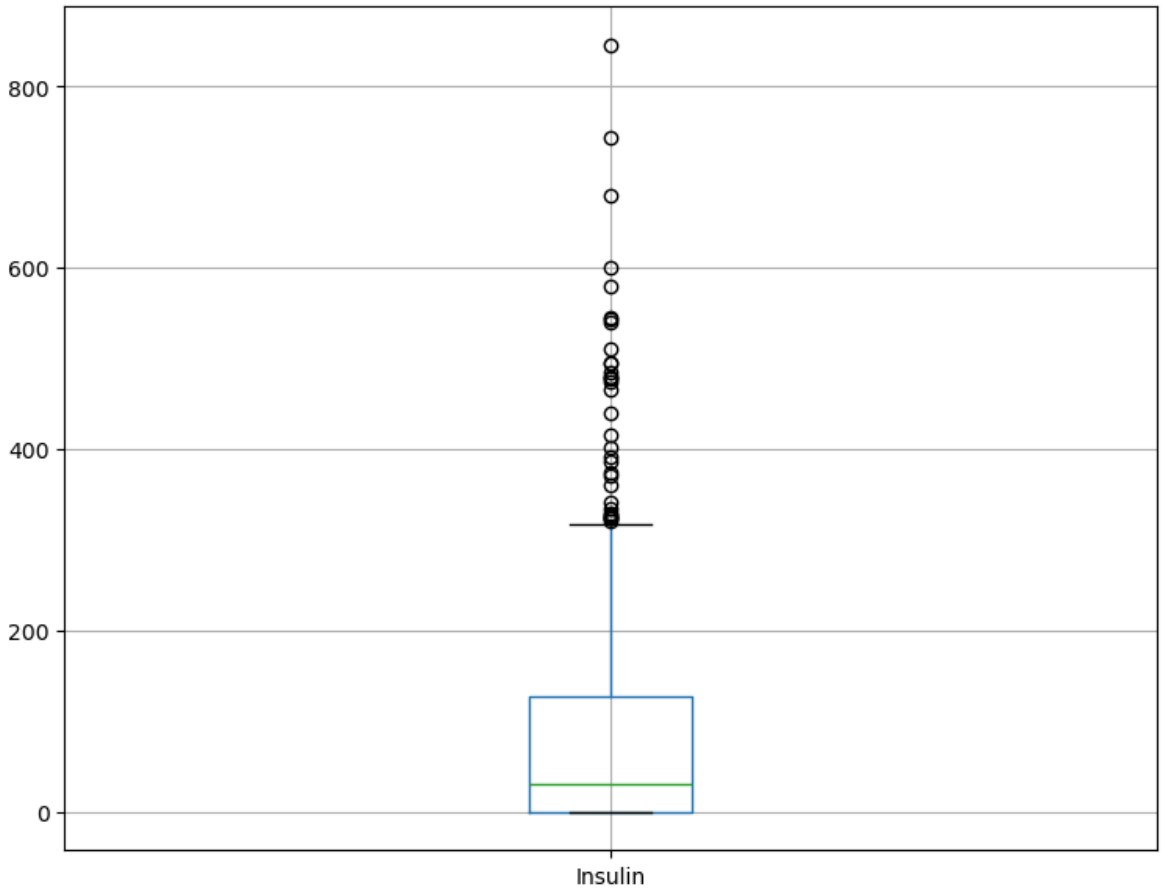
In [20]:
```
df.isnull().sum()
```

Out[20]:
```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [21]:
```
cols = ['Glucose','BloodPressure','SkinThickness','BMI','Age']
df[cols].boxplot(figsize=(12,6))
plt.title(" Diabetes Dataset")
plt.show()
```



In [22]:
```
col=['Insulin']
df[col].boxplot(figsize=(9,7))
plt.show()
```



In [23]:
```
cols = ['Glucose','BloodPressure','SkinThickness','Insulin','BMI']
for col in cols:df[col] = df[col].replace(0, np.nan)
df
```

Out[23]:

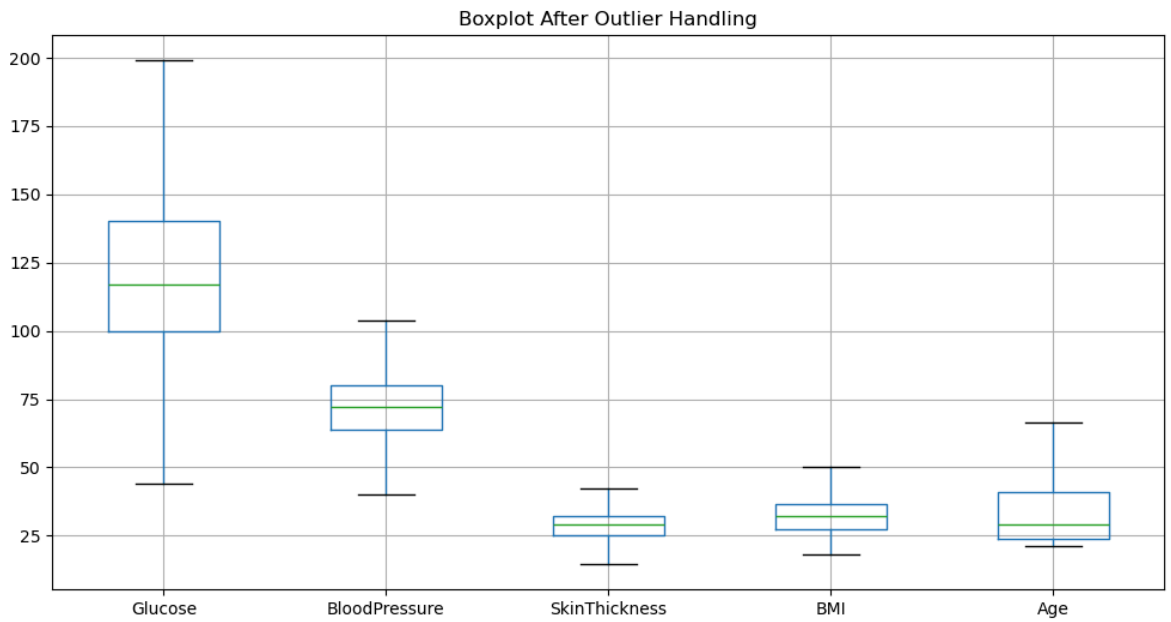
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPe
0	6	148.0	72.0	35.0	NaN	33.6	
1	1	85.0	66.0	29.0	NaN	26.6	
2	8	183.0	64.0	NaN	NaN	23.3	
3	1	89.0	66.0	23.0	94.0	28.1	
4	0	137.0	40.0	35.0	168.0	43.1	
...	...	...	...	...	...	...	
763	10	101.0	76.0	48.0	180.0	32.9	
764	2	122.0	70.0	27.0	NaN	36.8	
765	5	121.0	72.0	23.0	112.0	26.2	
766	1	126.0	60.0	NaN	NaN	30.1	
767	1	93.0	70.0	31.0	NaN	30.4	

768 rows × 9 columns

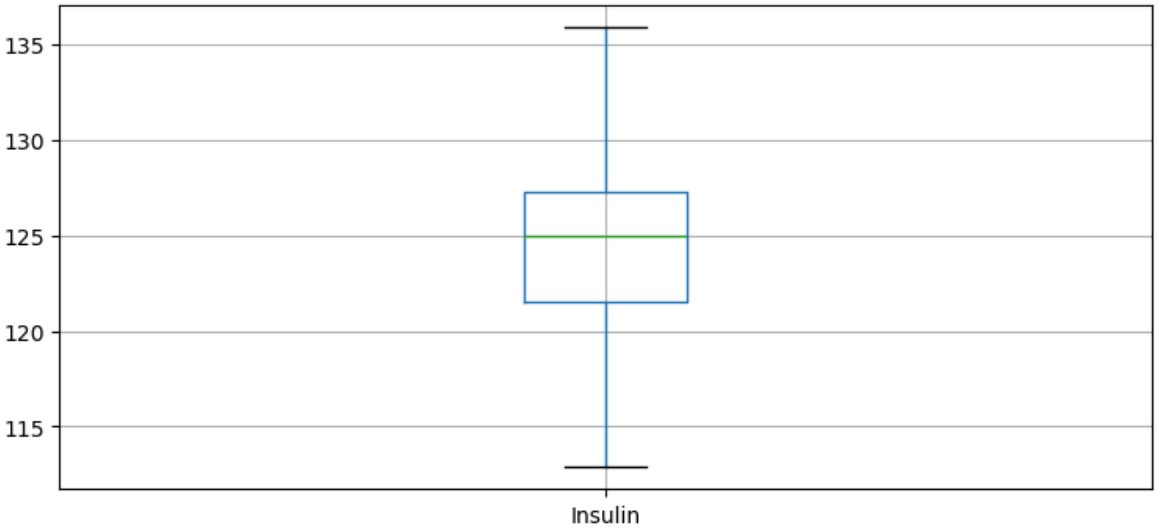
In [24]:
```
for col in cols: df[col] = df[col].fillna(df[col].median())
```

In [25]:
```
for col in df.columns[:-1]:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR
    df[col] = df[col].clip(lower, upper)
```

In [26]:
```
col= ['Glucose','BloodPressure','SkinThickness','BMI','Age']
df[col].boxplot(figsize=(12,6))
plt.title("Boxplot After Outlier Handling")
plt.show()
```



In [27]:
```
col=['Insulin']
df[col].boxplot(figsize=(9,4))
plt.show()
```



In [28]:
```
X = df.drop('Outcome', axis=1)
y = df['Outcome']
```

In [29]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

In [30]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [31]:
```
from sklearn.naive_bayes import GaussianNB
gaussian = GaussianNB()
gaussian.fit(X_train, y_train)
```

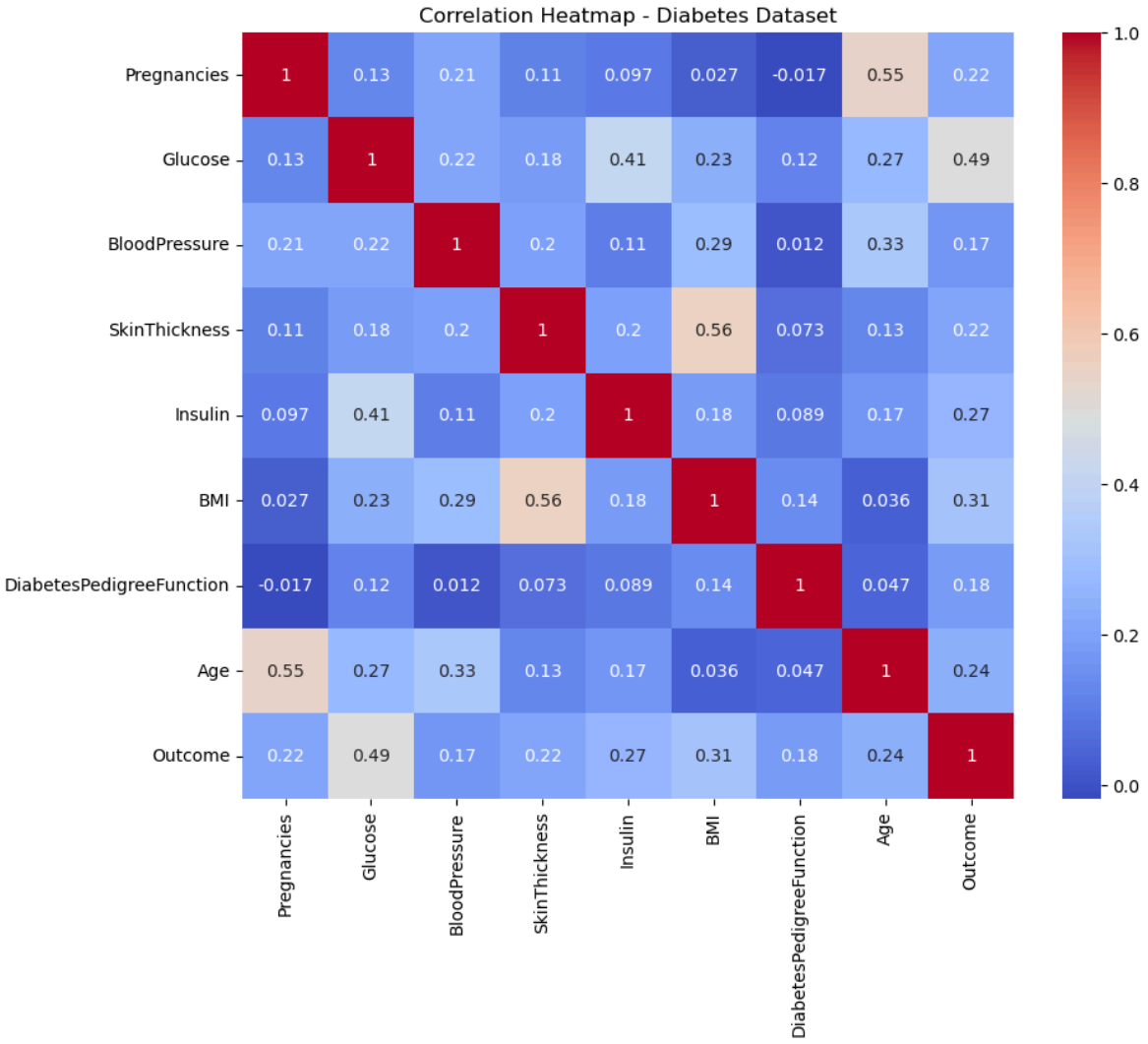
Out[31]:
```
▾ GaussianNB
▶ Parameters
```

In [32]:
```
Y_pred = gaussian.predict(X_test)
```

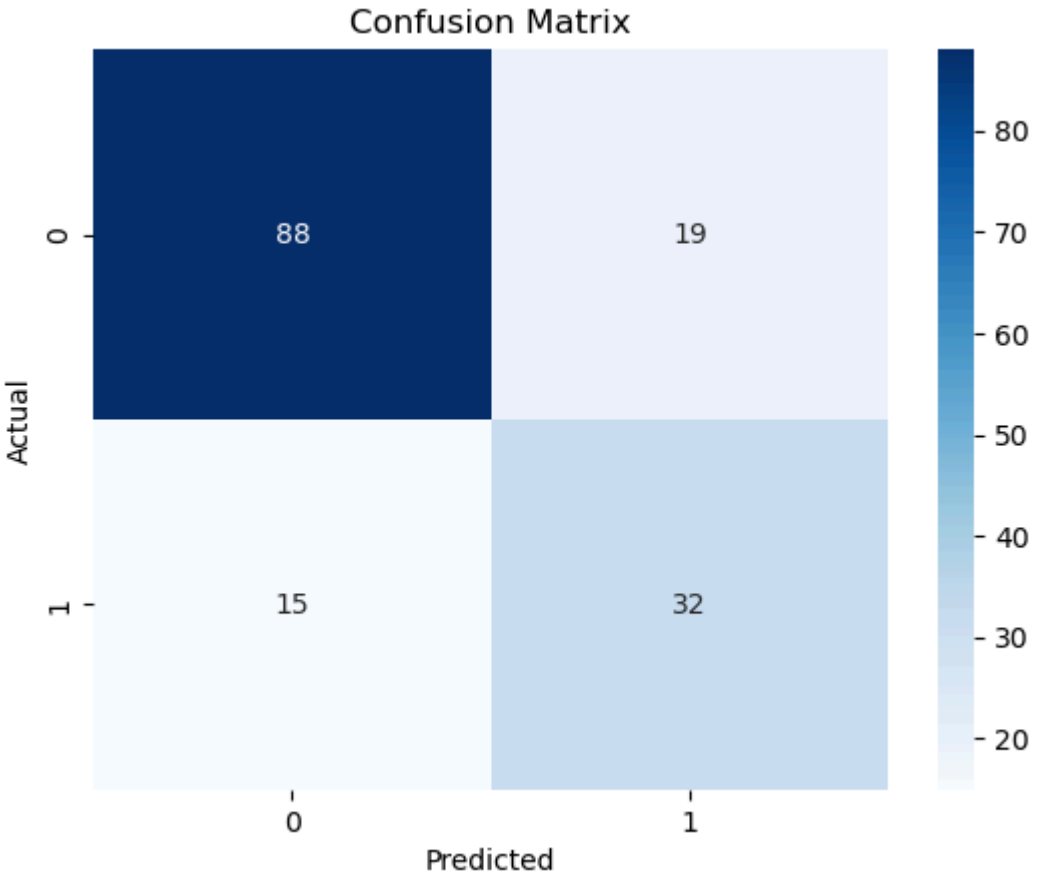
In [33]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score
accuracy = accuracy_score(y_test, Y_pred)
precision = precision_score(y_test, Y_pred)
recall = recall_score(y_test, Y_pred)
cm = confusion_matrix(y_test, Y_pred)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 0.7792207792207793
Precision: 0.6274509803921569
Recall: 0.6808510638297872
Confusion Matrix:
 [[88 19]
 [15 32]]
```

In [34]:
```
import seaborn as sns
corr = df.corr()
plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap - Diabetes Dataset")
plt.show()
```



In [35]:
```
import seaborn as sns
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```


Confusion Matrix
88
19
15
32
0
1
Actual
0
1
Predicted
80
70
60
50
40
30
20

In [ ]: